Implementation and Performance Evaluation of Vision Transformer Model Based on MCU-NPU(STM32N6)

https://github.com/minchoCoin/stm32n6-transformer

KICS winter conference 2026

Taehun Kim

bigteach0508@pusan.ac.kr

Pusan National University

School of AI

02.05.2026

Contents

- Introduction
- Methodology
  - NPU
  - ViT Model Design for NPU
  - Model Quantization
  - Model Compilation
- Experiments
  - Experiments Setting
  - Results
- Conclusion

Introduction

- On-device AI
  - Execute inference directly within edge devices or MCUs (Micro Controller Units)
  - The growing need for on-device AI due to the recent increase in IoT devices and edge computing devices[1]
  - Reduced latency, mitigated dependence on cloud computing resources and networks, and enhanced system stability

- MCU
  - A low-power processor integrating the CPU (Central Processing Unit), memory, and I/O ports onto a single chip
  - Optimized for resource-constrained systems
  - Limitations in operating AI models requiring high computational demands due to constrained processing power and memory capacity

Introduction (cont'd)

- NPU(Neural Processing Unit)
  - The NPU integrated into the MCU enables high-speed processing of complex neural network models.
  - Limited cases of running and validating Transformer-based models on NPUs embedded in MCUs released to date
    - Transformers are utilized in various fields such as vision, audio, and natural language processing.
    - The self-attention structure, which requires significant computational resources, poses challenges for implementation in existing MCU environments.

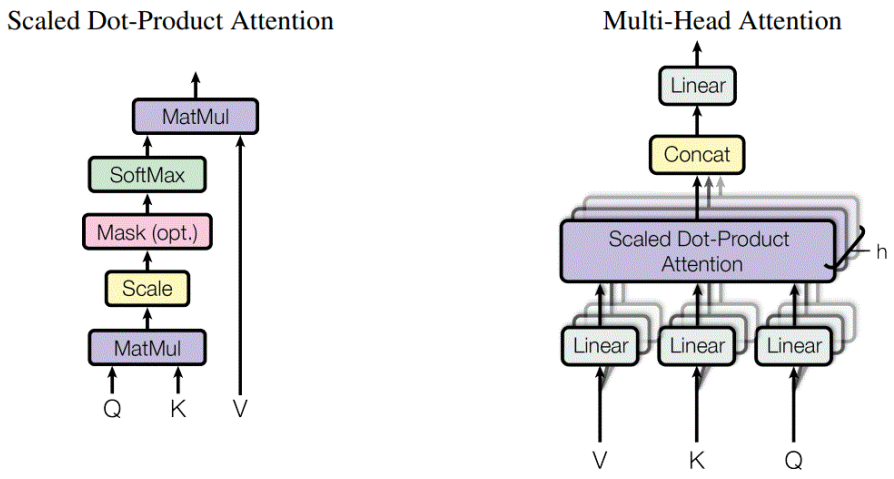


Figure 1. Transformer Architecture

† STMicroelectronics, AI:STM32Cube.AI model performances – stm32mcu, available at: https://wiki.st.com/stm32mcu/wiki/AI%3ASTM32Cube.AI_model_performances

NPU

- NPU
  - Using the STM32N6 MCU released by ST Microelectronics (STM)
    - ST Neural-ART Accelerator 14 NPU
    - Designed to accelerate quantized CNN models
    - Not all operations related to neural network models can be accelerated; those that cannot be accelerated are executed on the CPU.
      - To run neural network models on the NPU, model compilation is required using the STEdgeAI tool provided by STMicroelectronics.
      - Due to limitations in the operations supported for compilation based on the NPU design, it is necessary to construct the Transformer using supported operations.

Figure 2. STM32N6570-DK Evaluation Board for STM32N6

Table 1. Comparison of AI Computation Efficiency Between STM32N6(NPU) and STM32H7(MCU only)†

Model	Image size	Device	Inference Time	Power
MobileNet v2 1.0	224x224x3	**STM32N6 @1GHz**	**20.7ms (48fps)**	**7mJ (1.939μWh)**
Yamnet 1024	64x96	**STM32N6 @1GHz**	**9.7ms (103fps)**	**3.2mJ (0.886μWh)**
MobileNet v2 0.35	128x128x3	STM32H7 @400MHz	94.9ms (11fps)	20.4mJ (5.65μWh)
Yamnet 256	64x96	STM32H7 @400MHz	77.3ms (13fps)	16.3mJ (4.52μWh)

ViT Model Design for NPU

- ViT Model Design for NPU
  - Based on the existing Vision Transformer (ViT)[2] model, but with structural modification for deployment on an NPU.
  - Perform PE(Patch Embedding) during the preprocessing stage
  - When a model incorporates both PE and self-attention, model architecture analysis using STEdgeAI is not possible.

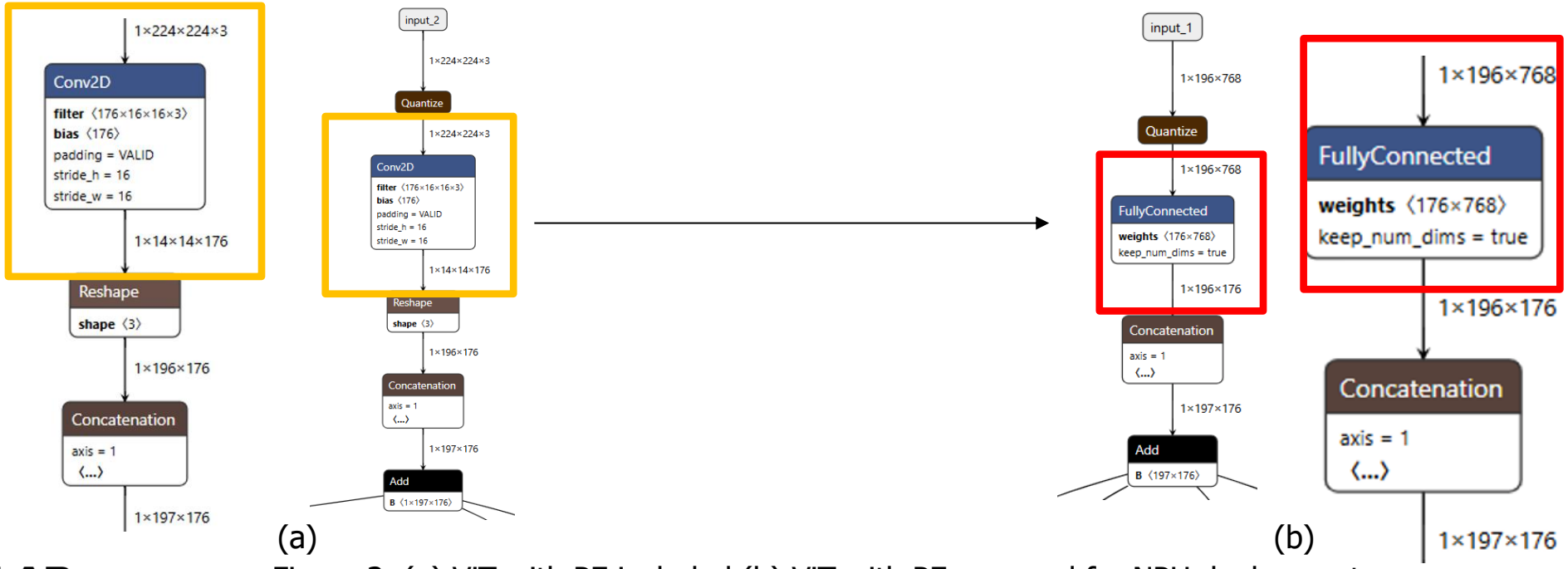


Figure 3. (a) ViT with PE included (b) ViT with PE removed for NPU deployment

ViT Model Design for NPU (cont'd)

- ViT Model Design for NPU
  - Using ReLU(Rectified Linear Unit) for activation function of Feed-forward MLP(Multi-Layer Perceptron)
  - GELU(Gaussian Error Linear Unit) of TFLite(Tensorflow Lite) is not supported on the NPU

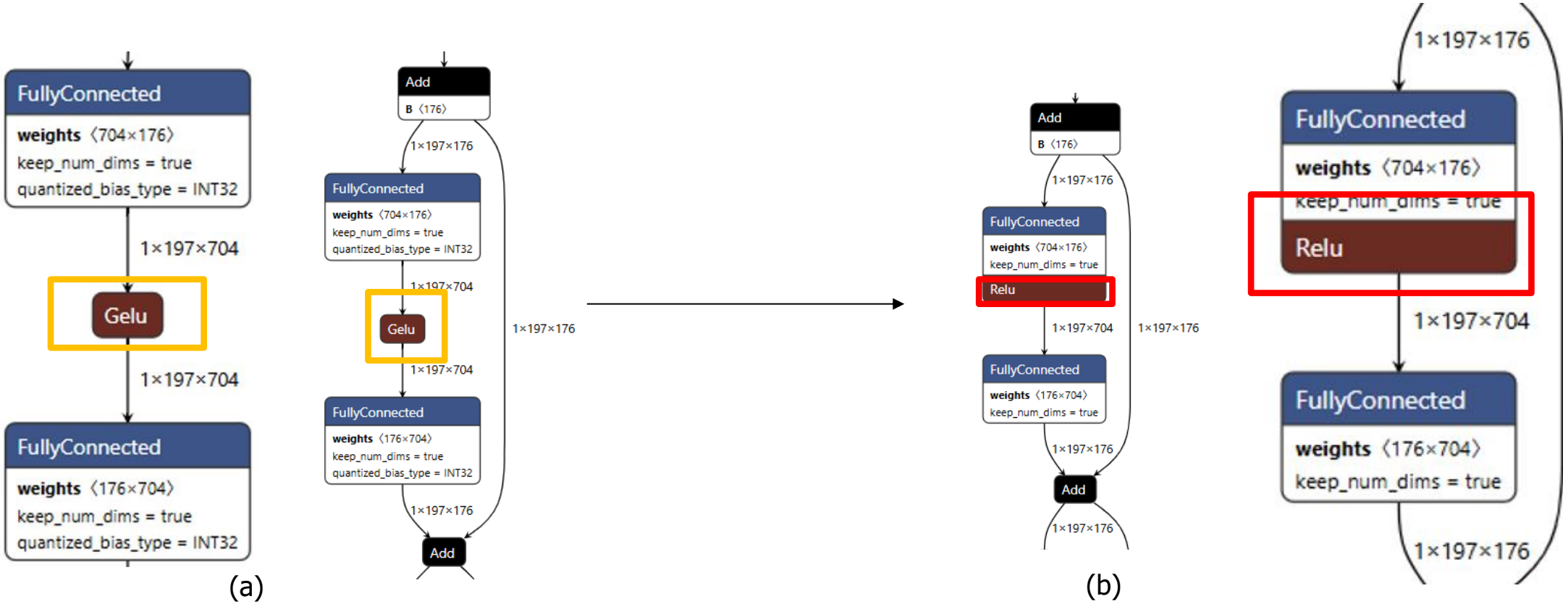
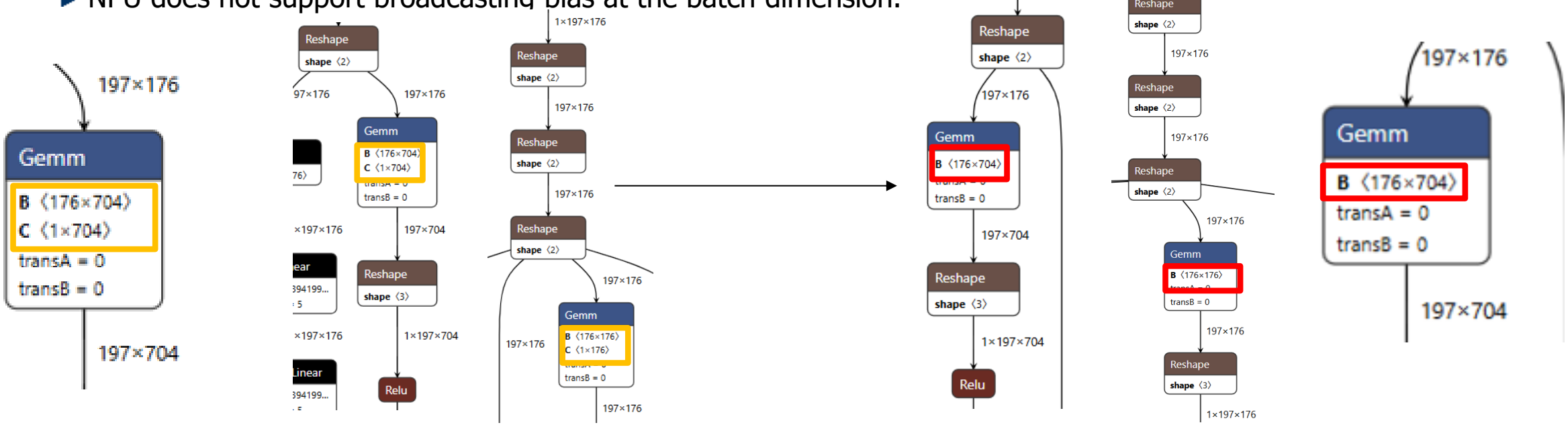


Figure 4. (a) ViT with GELU (b) ViT with GELU replaced by ReLU for NPU deployment

ViT Model Design for NPU (cont'd)

- Designing a ViT Model for NPU Supporting Only Fully Connected(FC) Layers Receiving 2D inputs
  - FC Layer Bias Removal
    - During compilation, remove the existing batch dimension from the input of the 3D FC layer
    - Remove the original batch dimension from the input $(1, patch, dim)$ and set the patch as the batch dimension, resulting in input of the form $(patch, dim)$
    - NPU does not support broadcasting bias at the batch dimension.

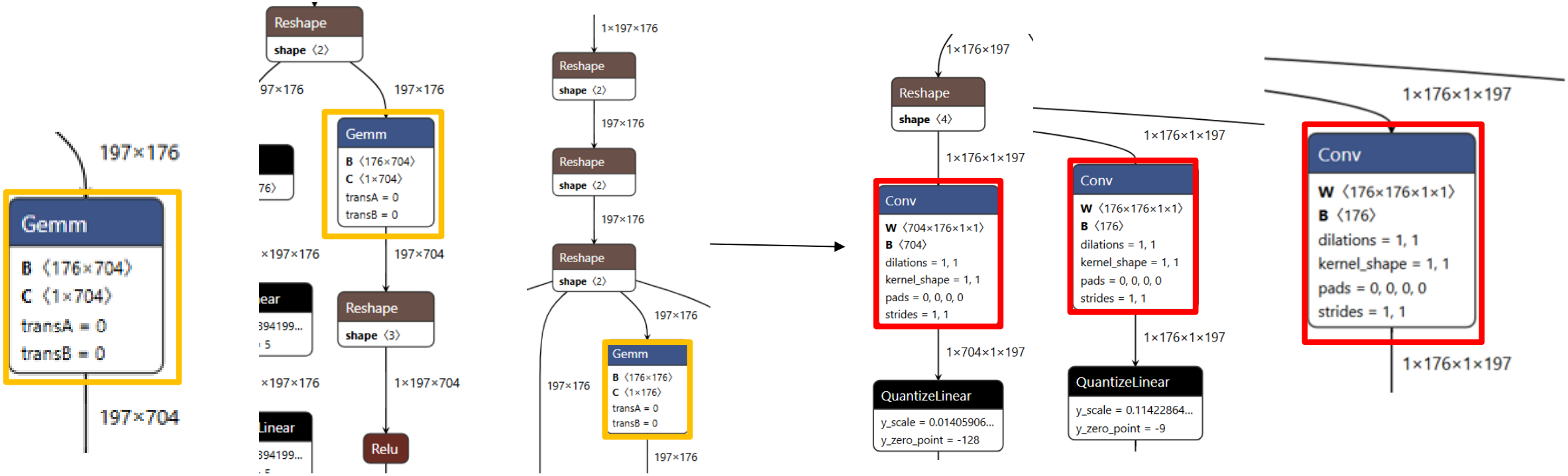


(a) (b)

Figure 5. (a) ViT with bias included in the FC layer (b) ViT with bias excluded in the FC layer for NPU deployment

ViT Model Design for NPU (cont'd)

- Designing a ViT Model for NPU Supporting Only FC Layers Receiving 2D inputs
  - Replace the FC layer with a 1x1 convolution layer that includes the same bias.
    - ViT with bias



(a) (b)

ESLAB Embedded System Lab Figure 6. (a) ViT with bias included in the FC layer (b) ViT with FC layer changed to 1x1 convolution for NPU deployment

Model Quantization

- Model Quantization
  - The NPU integrated into the STM32N6 MCU supports only models quantized to 8-bit integers (int8) and unsigned 8-bit integer (uint8) inputs.
  - Perform post-training quantization (PTQ) using TFLiteConverter
    - Model weights are set to int8 quantization, input is uint8, and output is float32.

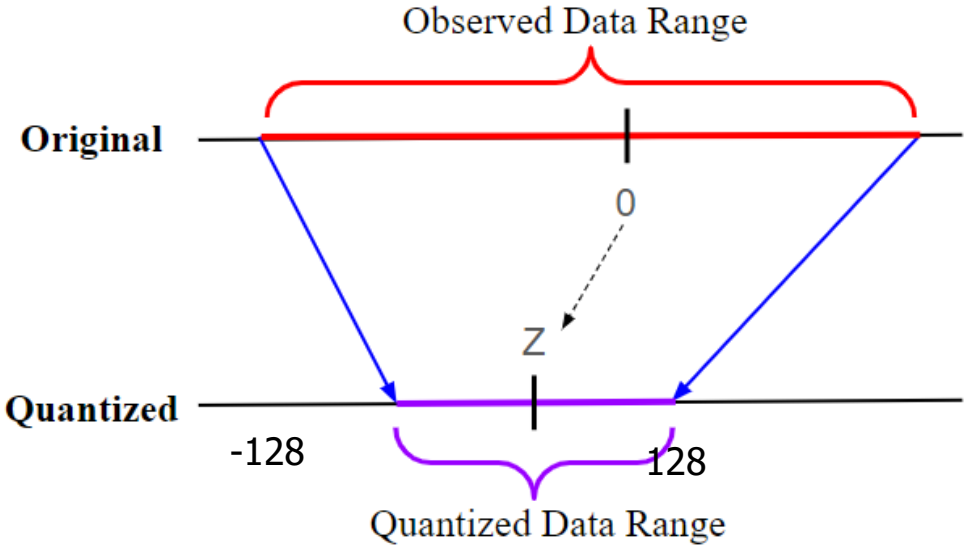


Figure 7. Model Quantization

Model Compile

- Model Compile with STEdgeAI
  - Convert TFLite or Open Neural Network Exchange (ONNX) files into ONNX files containing the model structure and JavaScript Object Notation (JSON) containing weight information, then compile them.
  - During compilation, operations are converted into hardware instructions, assigned to the NPU and CPU, and weights are placed in memory.
  - After compilation, the following files are generated: network.c containing the model structure, network_data.bin containing the model weights, and network_ecblobs.h containing the instructions to be input to the NPU.
- FC operations and 1x1 convolutions are assigned to the NPU, while the matmul operator is assigned to the CPU.

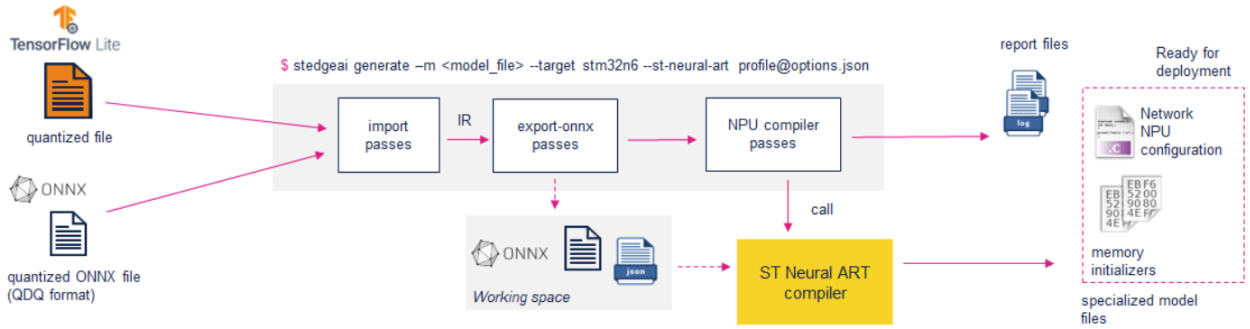


Figure 8. Model Compile with STEdgeAI

Model Compile(cont'd)

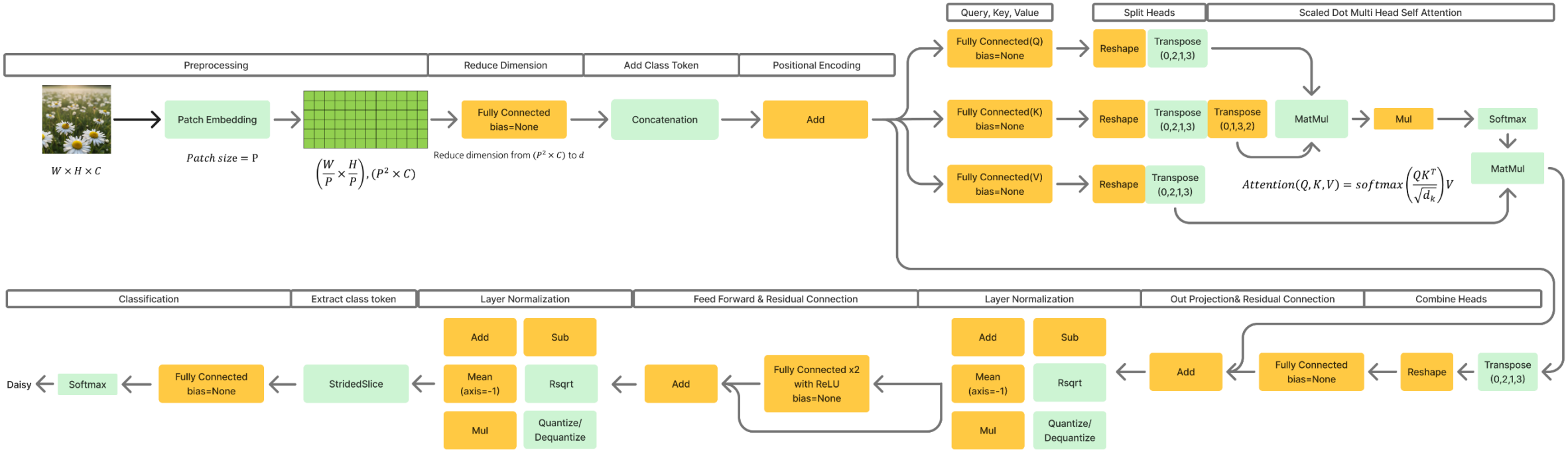


Figure 9. CPU and NPU allocation details for operators used in the ViT model. The PE is performed in preprocessing stage. Operators marked in orange run on the NPU, while those marked in light green run on the CPU. Models incorporating bias parameters replace all FC layers except the one used for classification with 1x1 convolutions containing bias parameters.

Experiment Setting

- To evaluate inference time with and without NPU acceleration, four types of ViT designs
- ViT-T(Tiny), ViT-S(Small), ViT-B(Base), ViT-L(Large)
  - w/o bias refers to an unbiased(FC) model, w/bias refers to a biased(1x1 convolution) model

Table 2. Vision Transformer Architecture for NPU deployment

	ViT-T	ViT-S	ViT-B	ViT-L
Image	96	160	160	224
Patch	16	16	16	16
Hidden size	128	128	144	176
Layers	4	6	6	6
Heads	4	4	4	8
MLP size	128	256	576	704
Params(w/o bias) (M)	0.494	0.889	1.608	2.371
MACs(w/o bias) (M)	20.3	113.4	188.0	609.5
Params(w/ bias) (M)	0.498	0.894	1.616	2.380
MACs (w/ bias) (M)	20.4	113.6	188.4	610.5

† ai planet, Data Sprint #25: Flower Recognition, from 20 Feb 2021 to 02 Mar 2021, available at: https://aiplanet.com/challenges/61/data-sprint-25-flower-recognition-61/overview/about

Experiment Setting (cont'd)

- Experiment for Evaluating Model Inference Time
  - STM32N6570-DK development board for the STM32N6 that integrated CPU and NPU
  - NUCLEO-H753ZI development board for the STM32H7, featuring only a CPU inside the MCU
- Training
  - The training was conducted on a PC, and the dataset used was the image dataset consisting of five types of fresh flowers, which was used in the Data Sprint #25: Flower Recognition† challenge held on AI Planet

Table 3. Device Specification

	STM32N6570-DK	NUCLEO-H753ZI
CPU	Arm Cortex M55@800MHz 1352DMIPS@800MHz	Arm Cortex M7@480MHz 1027DMIPS@480MHz
NPU	ST Neural-ART Accelerator@1GHz 600GOPS@1GHz	-
Flash Memory	128MB Octo-SPI flash memory @200MHz	2MB@240MHz
memory	NPU RAM 448kB x4 @900MHz	1MB@240MHz
	System RAM 1MB+624kB+400kB@400MHz	
	32MB Hexadeca-SPI PSRAM@200MHz	

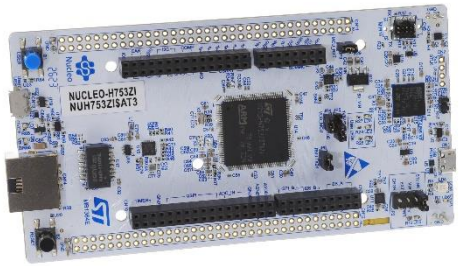


Figure 9. NUCLEO-H753ZI development board for the STM32H7

Results

- Comparative analysis of model inference time by device
  - Compared to MCUs without an NPU, MCUs with an NPU achieve an average reduction of approximately 91% in inference time during inference.
  - Operators such as FC and 1x1 convolutions used in ViT are accelerated in parallel Compared to ViT-B, ViT-L exhibits a greater increase in inference time on NPU boards relative to its increased computational load.
    - The computational load of self-attention operations executed on the CPU increases significantly as the number of patches increases, proportional to the square of the number of patches.
    - Demonstrates that NPU can further reduce inference time when supporting self-attention.
  - The ViT-L model, which exceeds 1MB of memory usage, cannot be deployed on the NUCLEO-H753ZI board due to its limited memory capacity.

Table 4. Comparative analysis of ViT model inference time by device

	board	ViT-T	ViT-S	ViT-B	ViT-L
Inference time(ms) (w/o bias)	STM32N6	**12**	**72**	**82**	**417**
	H753ZI	142	748	968	-
Inference time(ms) (w/ bias)	STM32N6	**9**	**61**	**71**	**373**
	H753ZI	108	632	826	-

Results (cont'd)

- Model inference demonstration

Figure 10. Vision Transformer model inference on STM32N6

Conclusion

- Verification of ViT Acceleration Potential in MCUs with Embedded NPUs
- Performed model architecture modification and lightweighting to optimize the existing ViT structure for the NPU.
- Through experimentation, we verified that ViT achieves an average reduction of approximately 91% in inference time when executed on an NPU compared to when executed on a CPU.
- Future research will validate the real-time capability of on-device ViT models by comparing computational speeds with the Alif Ensemble E8 MCU, which incorporates the Ethos U85 NPU supporting ViT model self-attention operations.

References

- [1] Wang, Xubin, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. "Empowering edge intelligence: A comprehensive survey on on-device ai models." ACM Computing Surveys 57, no. 9 (2025): 1-39.
- [2] Dosovitskiy, Alexey. "An image is worth 16x16 words: Transformers for image recognition at scale." arXiv preprint arXiv:2010.11929 (2020).

Appendix

- Error message of STEdgeAI with original ViT compile

```
D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model>stedgeai genera
te --model custom_vit_int8_enc.tflite --target stm32n6 --st-neural-art default@user_neuralart_STM32N6570-DK.json
ST Edge AI Core v2.2.0-20266 2adc00962

INTERNAL ERROR: Exported ONNX could be malformed since ONNX shape inference fails

D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model>_
```

Figure 11. Error message when ViT model contains Patch Embedding

```
D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model>stedgeai genera
te --model err2.tflite --target stm32n6 --st-neural-art default@user_neuralart_STM32N6570-DK.json
ST Edge AI Core v2.2.0-20266 2adc00962

NOT IMPLEMENTED: not implemented onnx layer: Nonlinearity  (nl_32)

D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model>_
```

Figure 12. Error message when ViT model contains TFLite GELU

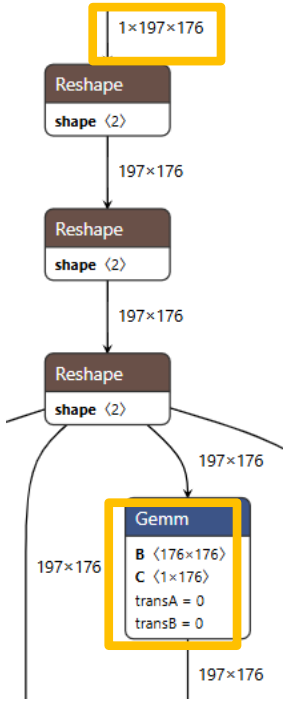
Appendix

- Error message of STEdgeAI with original ViT compile

```
C:₩Windows₩System32₩cmd.exe
2N6-GettingStarted-ImageClassification-main/Model/my_mpools/stm32n6-app2_STM32N6570-DK.mpool" --save-mpool-file "D:/dongye_2025/code_ppt/STM3
2N6-GettingStarted-ImageClassification-main/STM32N6-GettingStarted-ImageClassification-main/Model/st_ai_ws/neural_art__network/stm32n6-app2_S
TM32N6570-DK.mpool" --out-dir-prefix "D:/dongye_2025/code_ppt/STM32N6-GettingStarted-ImageClassification-main/STM32N6-GettingStarted-ImageCla
ssification-main/Model/st_ai_ws/neural_art__network/" -O3 --all-buffers-info --mvei --cache-maintenance --Oauto-sched --native-float --enable
-virtual-mem-pools --Omax-ca-pipe 4 --Ocache-opt --Os --enable-epoch-controller --output-info-file "c_info.json"

      >>> Shell execution has FAILED (returned code = 1)
      $ C:₩ST₩STEdgeAI₩2.2₩Utilities₩windows₩atonn.exe -i D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-Get
tingStarted-ImageClassification-main₩Model₩st_ai_output₩err1_OE_3_3_0.onnx --json-quant-file D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-I
mageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model₩st_ai_output₩err1_OE_3_3_0_Q.json -g network.c --load-mdesc C:₩
ST₩STEdgeAI₩2.2₩Utilities₩configs₩stm32n6.mdesc --load-mpool D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-
GettingStarted-ImageClassification-main₩Model₩my_mpools₩stm32n6-app2_STM32N6570-DK.mpool --save-mpool-file D:₩dongye_2025₩code_ppt₩STM32N6-Ge
ttingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassification-main₩Model₩st_ai_ws₩neural_art__network₩stm32n6-app2_STM32N6
570-DK.mpool --out-dir-prefix D:₩dongye_2025₩code_ppt₩STM32N6-GettingStarted-ImageClassification-main₩STM32N6-GettingStarted-ImageClassificat
ion-main₩Model₩st_ai_ws₩neural_art__network/ -O3 --all-buffers-info --mvei --cache-maintenance --Oauto-sched --native-float --enable-virtual-
mem-pools --Omax-ca-pipe 4 --Ocache-opt --Os --enable-epoch-controller --output-info-file c_info.json

      saving memory pool description "D:/dongye_2025/code_ppt/STM32N6-GettingStarted-ImageClassification-main/STM32N6-GettingStarted-ImageCla
ssification-main/Model/st_ai_ws/neural_art__network/stm32n6-app2_STM32N6570-DK.mpool"
      Error: optimize_round /c/local/jenkins_cloud/workspace/2-STEDGEAI_BuildAtonnExe_Win/git/onnx_backend/platform_passes/transform_gemm_fc_
into_conv.cc:203: runTransform: Assertion `(b_shape.size() == 1) || ((b_shape[0].dim == M) || b_shape[0].dim == N)` failed.
      Warning: Missing Quantization info for Gemm_27_weights; will consider Gemm_27_weights as a native Float
      Warning: Missing Quantization info for Reshape_38_out_0; will consider Reshape_38_out_0 as a native Float
      Warning: Missing Quantization info for Gemm_40_weights; will consider Gemm_40_weights as a native Float
      Warning: Missing Quantization info for Reshape_39_out_0; will consider Reshape_39_out_0 as a native Float
      Warning: Missing Quantization info for Gemm_40_weights; will consider Gemm_40_weights as a native Float
      Warning: Missing Quantization info for Gemm_44_weights; will consider Gemm_44_weights as a native Float
      Warning: Missing Quantization info for Reshape_39_out_0; will consider Reshape_39_out_0 as a native Float
      Warning: Missing Quantization info for Gemm_44_weights; will consider Gemm_44_weights as a native Float
      Warning: Missing Quantization info for Transpose_47_out_0; will consider Transpose_47_out_0 as a native Float
```



M=197
N=176
b_shape=(1,176)

Figure 13. Error message when ViT model contains Fully Connected layer with bias